Aminah Ali

EBNF SYNTAX DIAGRAMS

BNF

1. **Backus-Naur Form** (BNF): A *metalanguage*, BNF is widely used as a notation for the grammars of computer programming languages, command sets and communication protocols, as well as a notation for representing parts of natural language grammars.

   BNF is the most general representation scheme.

Example:

```
<syntax> ::= <rule>
```

We use symbols like **:=, |, <,>, (,)** etc.

The main operators here are:

1. Alteration "|"
2. Concatenation ."
3. Grouping "( )"
4. Production Sign " → " or "**::=**"

For example **A:=** α | β and **B**:= **a** γ | **b** γ is in BNF.

```
<Expr> ::= <Term> | <Term> “+” <Expr>
<Term> ::= <Fact> | <Fact> “*” <Term>
<Fact> ::= <num> | <id> | “(“ <Expr> “)”
<id> ::= “x” | “y” | “z”
<num> ::= <digit> | <digit> <num>
<digit> ::= “0”|“1”|“2”|“3”|“4”|”5”|“6”|“7”|“8”|“9”
```

EBNF

EBNF Operator	Meaning
`{ X }`	Zero or more repetitions of X
`[ X ]`	Zero or one occurrence of X (optional)
`( X )`	Grouping

SIMPLE CONVERSION

CFG Pattern	EBNF Equivalent	Notes
`A → A x | x`	`A = x { x }`	Left recursion → iteration
`A → x A | x`	`A = x { x }`	Right recursion → iteration
`A → x A | ε`	`A = { x }`	Nullable right recursion
`A → B C | B`	`A = B [ C ]`	Optional tail
`A → p | q, used once`	Inline `(p | q)`	Eliminate single-use rule
`A → B ',' A | B`	`A = B { ',' B }`	Separated list
`A → x y | x`	`A = x [ y ]`	Optional element at end

$$exp \rightarrow exp\ addop\ term \mid term$$
$$addop \rightarrow + \mid -$$
$$term \rightarrow term\ mulop\ factor \mid factor$$
$$mulop \rightarrow *$$
$$factor \rightarrow (\ exp\ ) \mid \mathbf{number}$$

$$exp \rightarrow term \{ addop\ term \}$$
$$addop \rightarrow + \mid -$$
$$term \rightarrow factor \{ mulop\ factor \}$$
$$mulop \rightarrow *$$
$$factor \rightarrow (exp) \mid \mathbf{number}$$

statement → *if-stmt* | **other**

if-stmt → **if** **(** *exp* **)** *statement*

| **if** **(** *exp* **)** *statement* **else** *statement*

exp → **0** | **1**

statement → *if-stmt* | **other**

if-stmt → **if** **(** *exp* **)** *statement* [**else** *statement*]

exp → **0** | **1**

```
<Expr> ::= <Term> | <Term> “+” <Expr>
<Term> ::= <Fact> | <Fact> “*” <Term>
<Fact> ::= <num> | <id> | “(“ <Expr> “)”
<id> ::= “x” | “y” | “z”
<num> ::= <digit> | <digit> <num>
<digit> ::= “0”|“1”|“2”|“3”|“4”|”5”|“6”|“7”|“8”|“9”
```

```
Expr = Term, {“+”, Term};
Term = Fact, {“*”, Fact};
Fact = num | id | “(“, Expr, “)”;
id = “x” | “y” | “z”;
num = digit, {digit};
digit = “0”|“1”|“2”|“3”|“4”|“5”|“6”|“7”|“8”|“9”;
```

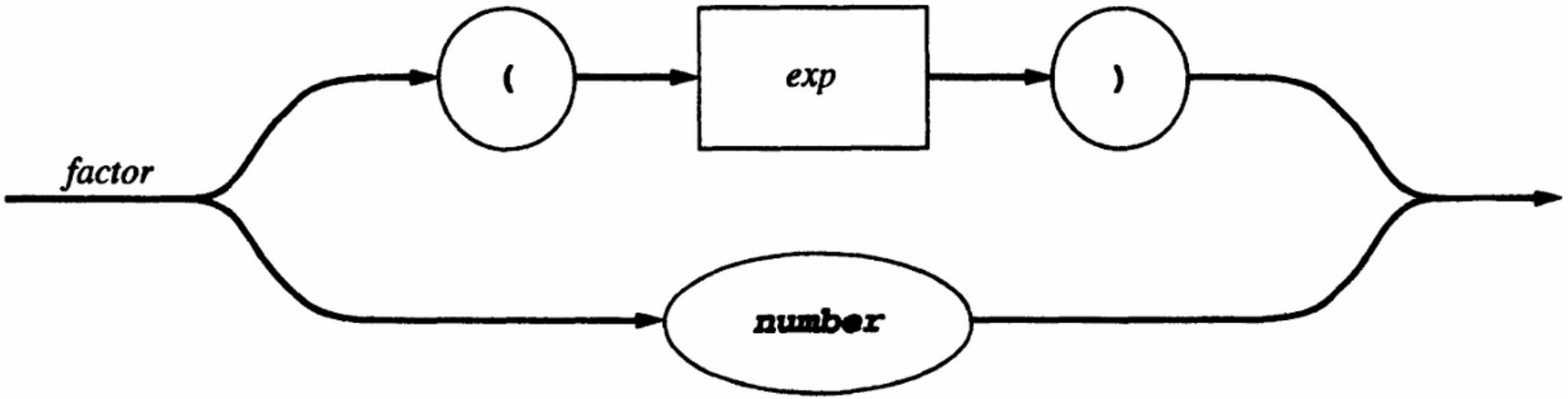
3.5.2 Syntax Diagrams

Graphical representations for visually representing EBNF rules are called **syntax diagrams**. They consist of boxes representing terminals and nonterminals, arrowed lines representing sequencing and choices, and nonterminal labels for each diagram representing the grammar rule defining that nonterminal. A round or oval box is used to indicate terminals in a diagram, while a square or rectangular box is used to indicate nonterminals.

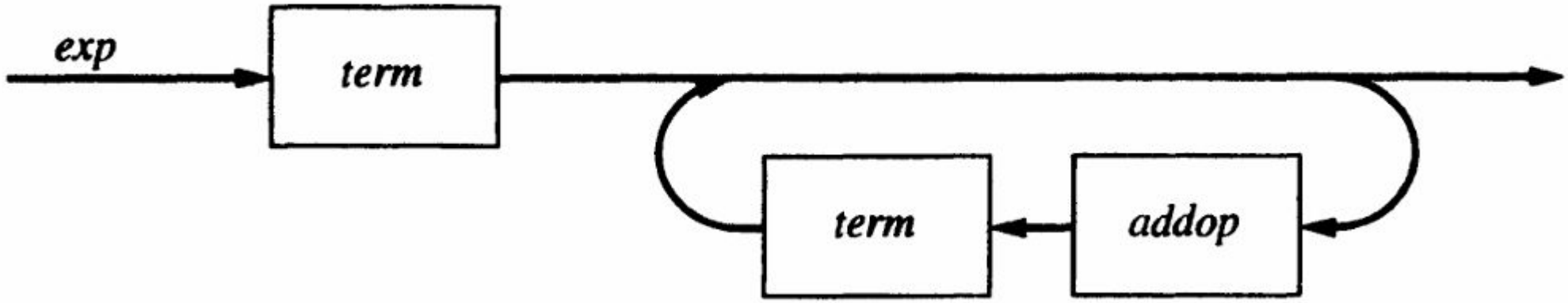
As an example, consider the grammar rule

$$\textit{factor} \rightarrow \texttt{(}\ \textit{exp}\ \texttt{)} \mid \textbf{number}$$

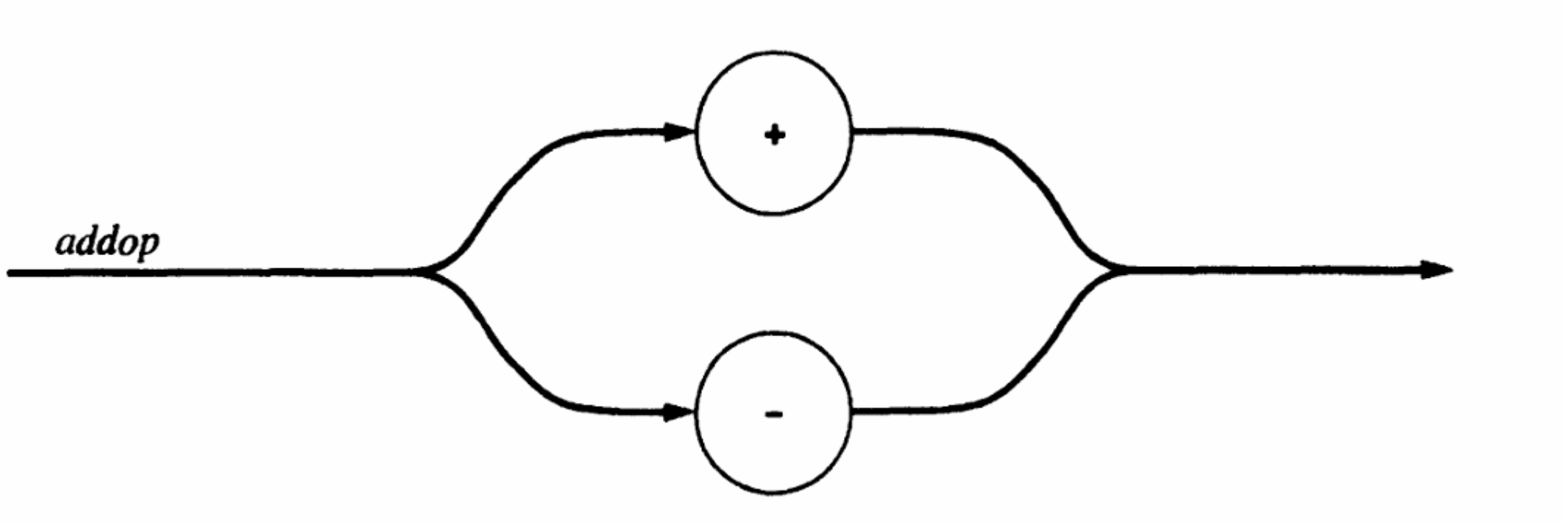
This is written as a syntax diagram in the following way:



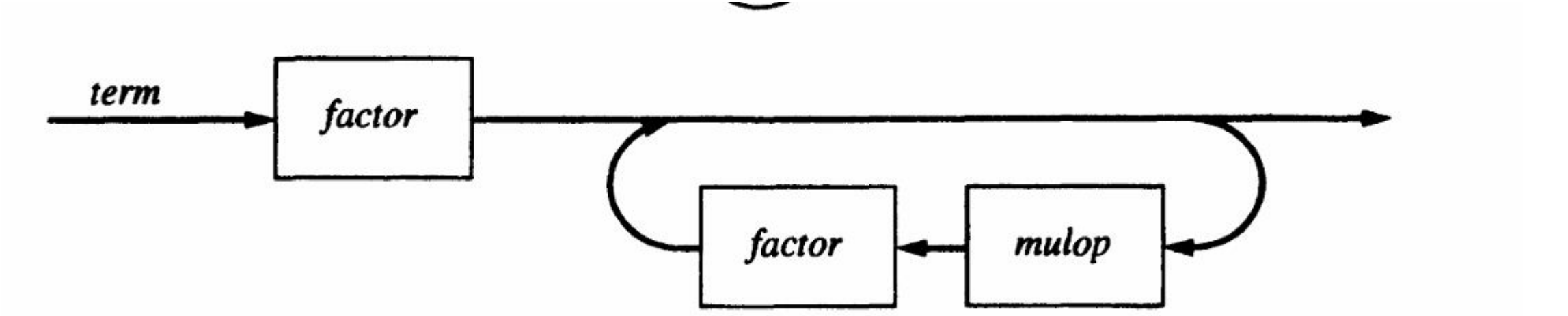
$$exp \rightarrow term \{ addop\ term \}$$

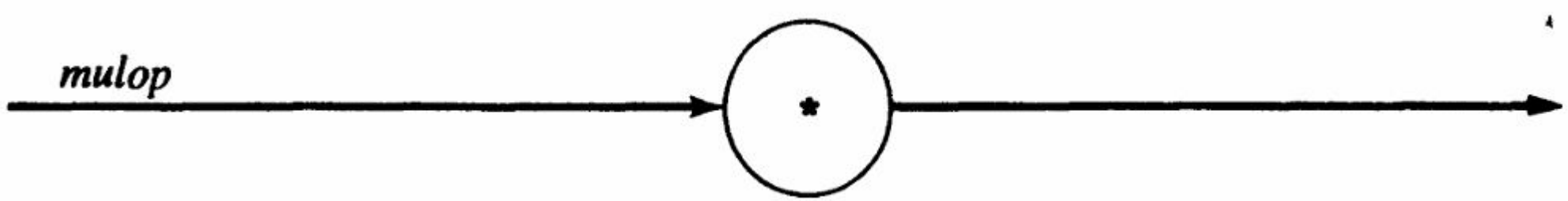


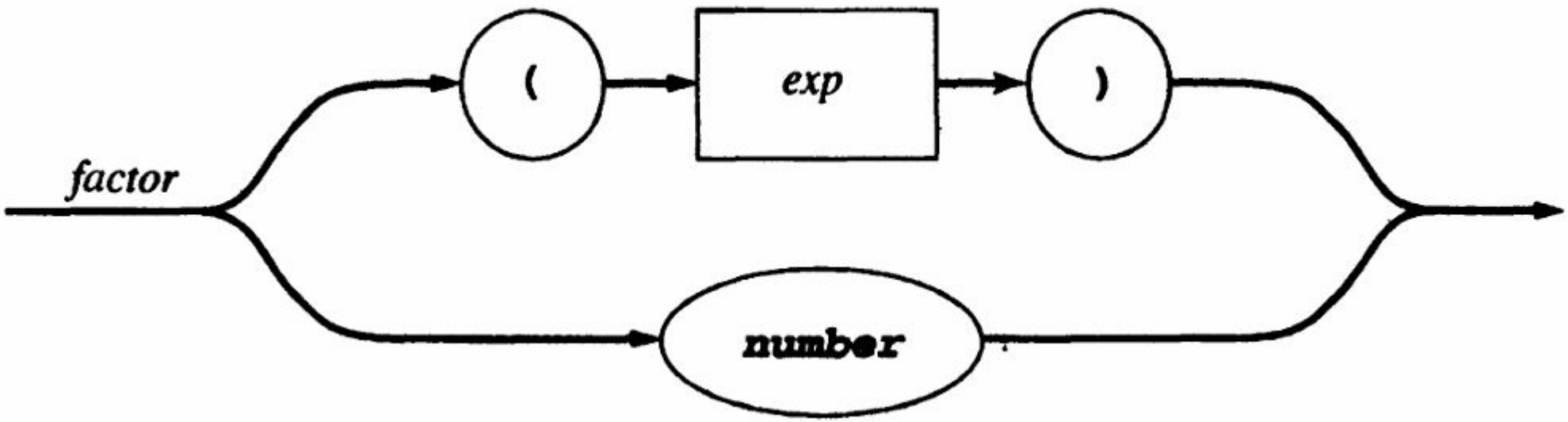
$$exp \rightarrow term \{ addop\ term \}$$
$$addop \rightarrow + \mid -$$
$$term \rightarrow factor \{ mulop\ factor \}$$
$$mulop \rightarrow *$$
$$factor \rightarrow (exp) \mid \mathbf{number}$$


addop
+
-

$$term \rightarrow factor \{ mulop\ factor \}$$



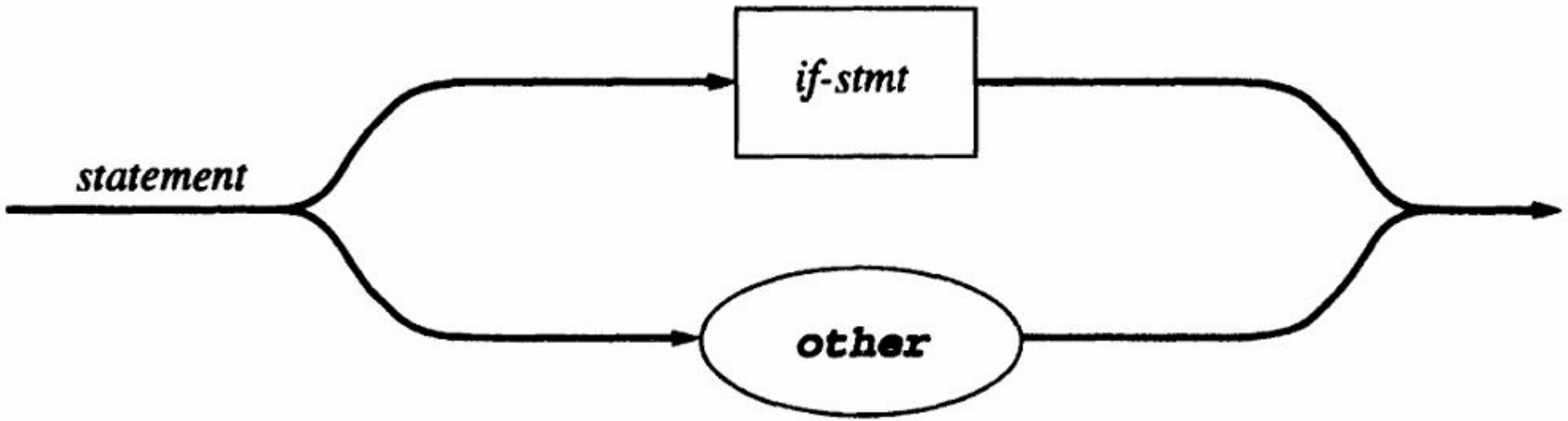

mulop
*

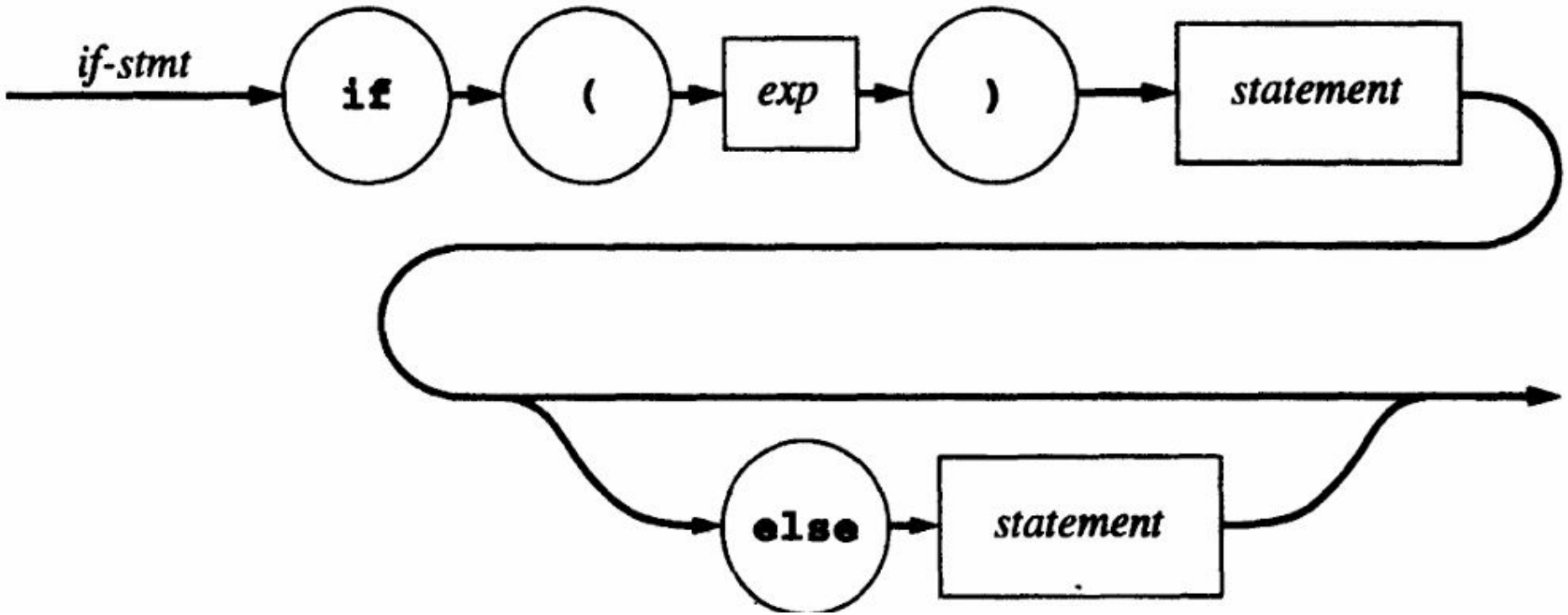

factor
(
exp
)
number

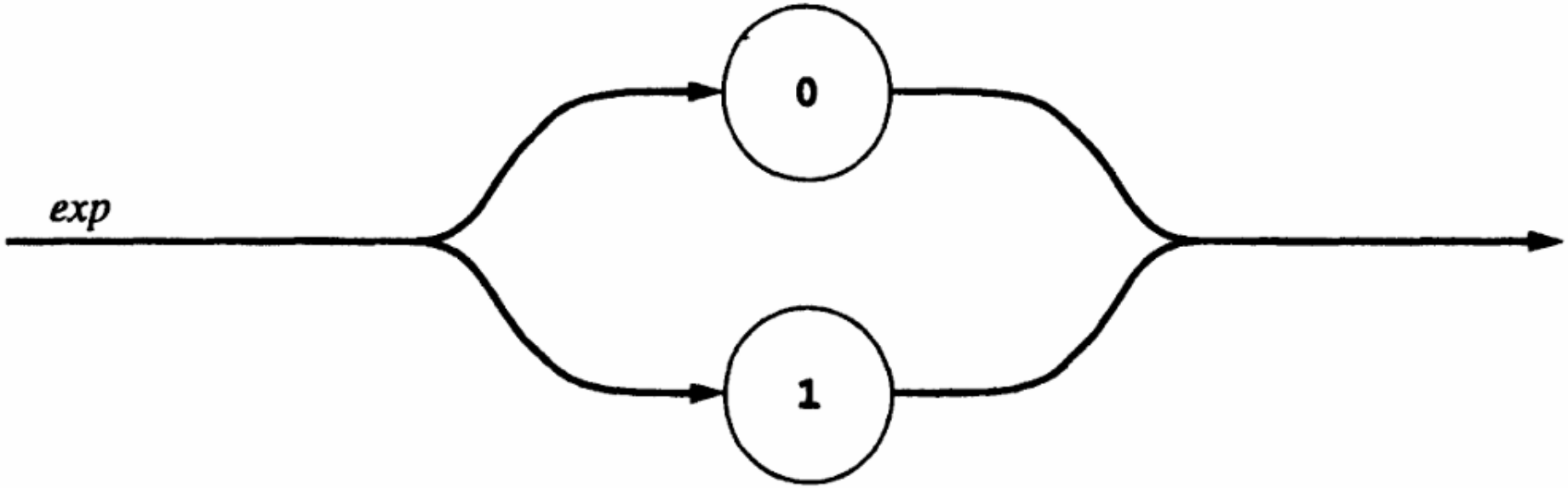
statement → *if-stmt* | **other**

if-stmt → **if** **(** *exp* **)** *statement* [**else** *statement*]

exp → **0** | **1**

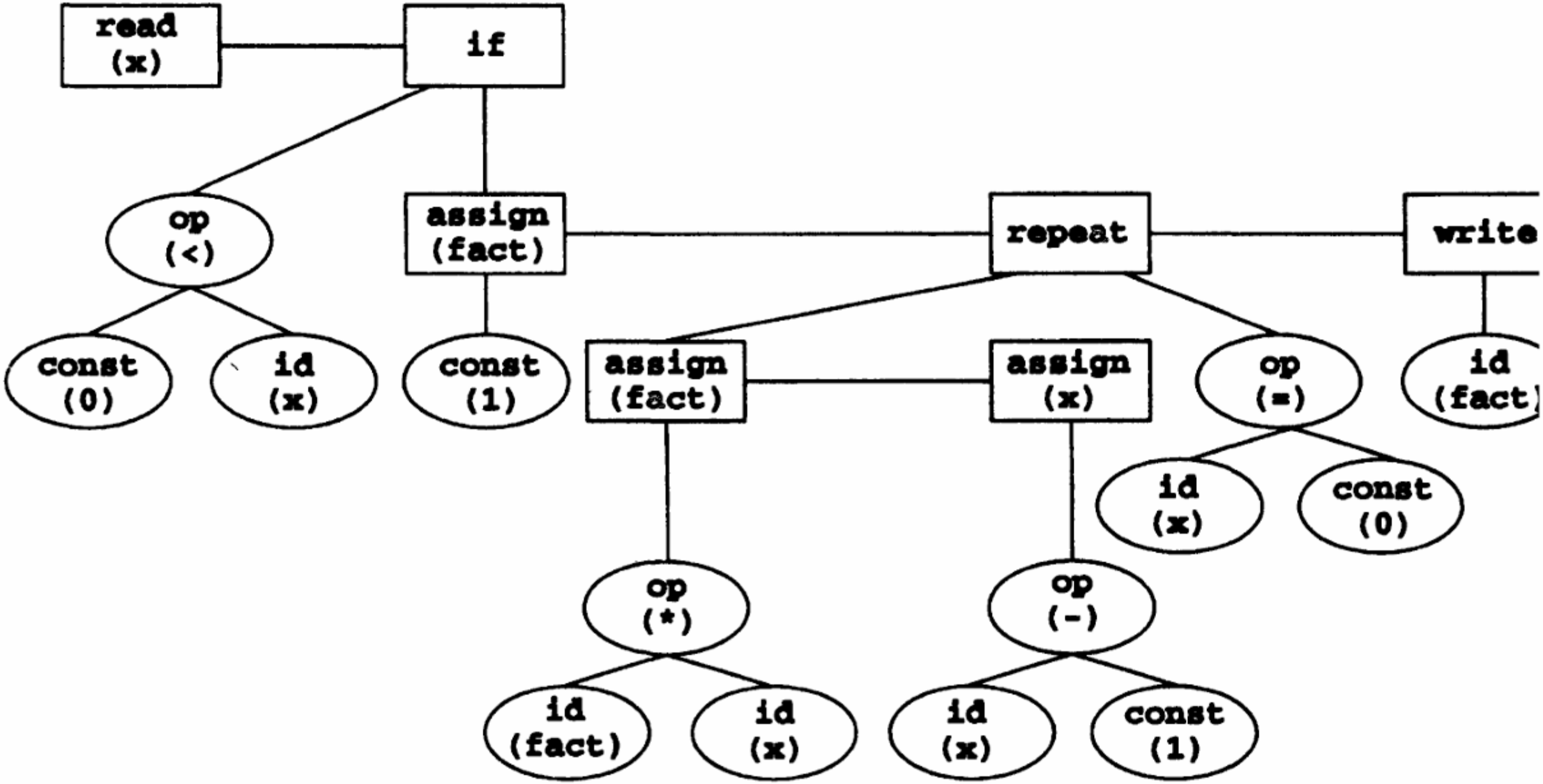

statement
if-stmt
other


if-stmt
if
(
exp
)
statement
else
statement


exp
0
1

TINY COMPILER

program → *stmt-sequence*
stmt-sequence → *stmt-sequence* **;** *statement* | *statement*
statement → *if-stmt* | *repeat-stmt* | *assign-stmt* | *read-stmt* | *write-stmt*
if-stmt → **if** *exp* **then** *stmt-sequence* **end**
| **if** *exp* **then** *stmt-sequence* **else** *stmt-sequence* **end**
repeat-stmt → **repeat** *stmt-sequence* **until** *exp*
assign-stmt → **identifier** **:=** *exp*
read-stmt → **read** **identifier**
write-stmt → **write** *exp*
exp → *simple-exp comparison-op simple-exp* | *simple-exp*
comparison-op → **<** | **=**
simple-exp → *simple-exp addop term* | *term*
addop → **+** | **-**
term → *term mulop factor* | *factor*
mulop → ***** | **/**
factor → **(** *exp* **)** | **number** | **identifier**


read
(x)
if
op
(<)
assign
(fact)
repeat
write
const
(0)
id
(x)
const
(1)
assign
(fact)
assign
(x)
op
(=)
id
(fact)
id
(x)
const
(0)
op
(*)
op
(-)
id
(fact)
id
(x)
id
(x)
const
(1)

```
program      → stmt_list
stmt_list    → stmt_list stmt | stmt
stmt         → assign_stmt | if_stmt | while_stmt | block
assign_stmt  → id '=' expr ';'
if_stmt      → 'if' '(' expr ')' stmt
             | 'if' '(' expr ')' stmt 'else' stmt
while_stmt   → 'while' '(' expr ')' stmt
block        → '{' stmt_list '}'
expr         → expr '+' term | expr '-' term | term
term         → term '*' factor | term '/' factor | factor
factor       → '(' expr ')' | id | number
```

3.8 Left-Factoring

Sometimes we find common prefix in many productions like $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3$, where α is common prefix. While processing α we cannot decide whether to expand A by $\alpha\beta_1$ or by $\alpha\beta_2$. So this needs back tracking. To avoid such problem, grammar can be left factoring.

Left factoring is a process of factoring the common prefixes of the alternatives of grammar rule. If the production of the form $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3$ has α as common prefix, by left factoring we get the equivalent grammar as

$$A \rightarrow \alpha A'$$
$$A' \rightarrow \beta_1 \mid \beta_2 \mid \beta_3$$

So, we can immediately expand A to $\alpha A'$.

Left-Factoring – Algorithm

- For each nonterminal A with two or more alternatives (production rules) with a common nonempty prefix, let us say

$$A \rightarrow \alpha\beta_1 \mid \ldots \mid \alpha\beta_n \mid \gamma_1 \mid \ldots \mid \gamma_m$$

Convert it into

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \ldots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \ldots \mid \beta_n$$

Left-factor the following grammar

$$A \rightarrow \underline{a}bB \mid \underline{a}B \mid cdg \mid cdeB \mid cdfB$$

Left-factor the following grammar

$$A \rightarrow ad \mid a \mid ab \mid abc \mid b$$

Q & A

Panel discussion session